Seminarski rad

# Single-linkage hijerarhijsko klasterovanje pomoću Primovog algoritma

Implementacija u C++ i demonstracija na spiralnom skupu podataka


| | |
|---|---|
| Autor: | Stefan Shrestha |
| Predmet: | Dizajn i analiza algoritama |
| Jezik i implementacija: | C++ |

# Sadržaj

**1 Uvod** **2**

**2 Single-linkage klasterovanje** **2**
2.1 Definicija . . . 2
2.2 Prednosti i ograničenja . . . 2

**3 Veza između single-linkage metode i MST-a** **2**
3.1 Grafovski model podataka . . . 2
3.2 Dobijanje klastera iz MST-a . . . 3
3.3 Izbor Primovog algoritma . . . 3

**4 Implementacija** **3**
4.1 Tok programa . . . 3
4.2 Strukture `Point` i `MstEdge` . . . 4
4.3 Klasa `DSU` . . . 4
4.4 `trim` . . . 4
4.5 `commaSplit` . . . 4
4.6 `tryDouble` . . . 4
4.7 `buildColumnMap` . . . 5
4.8 `resolveColumns` . . . 5
4.9 `readCSV` . . . 5
4.10 `normalize` . . . 5
4.11 `euclideanDistance` . . . 5
4.12 `primMST` . . . 5
4.13 `getClusters` . . . 6
4.14 `printClusterSummary` . . . 6
4.15 `saveCSV` . . . 6
4.16 `main` . . . 6

**5 Demonstracija programa na spiralnom skupu podataka** **6**
5.1 Skup podataka . . . 6
5.2 Rezultat . . . 6

**6 Složenost i memorija** **8**

**7 Zaključak** **8**

**Literatura** **8**

# 1 Uvod

Klasterovanje je postupak grupisanja objekata tako da objekti unutar iste grupe budu međusobno slični, dok se objekti iz različitih grupa što više razlikuju. Za razliku od klasifikacije, kod klasterovanja unapred nisu poznate klase koje algoritam treba da nauči. Zbog toga se ono ubraja u metode nenadgledanog učenja.

U ovom radu razmatra se single-linkage hijerarhijsko klasterovanje. Glavna ideja je da se tražena podela može dobiti pomoću minimalnog razapinjućeg stabla (MST). Time se problem grupisanja podataka povezuje sa klasičnim grafovskim algoritmima.

Implementirani program učitava podatke iz CSV fajla, bira osobine koje učestvuju u računanju rastojanja, normalizuje ih, gradi MST Primovim algoritmom i zatim izdvaja zadati broj klastera. Rezultat se čuva u fajlu `output.csv`.

Za demonstraciju algoritma korišćen je spiralni skup podataka. On je pogodan za single-linkage klasterovanje zato što sadrži nekonveksne, zakrivljene grupe koje ne odgovaraju pretpostavci da klasteri moraju biti približno kružni.

# 2 Single-linkage klasterovanje

## 2.1 Definicija

Neka su $A$ i $B$ dva klastera, a $d(x, y)$ rastojanje između tačaka $x$ i $y$. Single-linkage rastojanje između klastera definiše se kao

$$D(A, B) = \min_{x \in A,\ y \in B} d(x, y).$$

U svakom koraku hijerarhijskog postupka spajaju se dva klastera između kojih postoji najkraća veza. Na početku je svaka tačka zaseban klaster, a postupak se nastavlja sve dok se ne dobije željeni broj klastera ili dok se sve tačke ne spoje u jednu celinu.

## 2.2 Prednosti i ograničenja

Single-linkage ne zahteva da sve tačke istog klastera budu međusobno bliske. Dovoljno je da postoji lanac kratkih veza koji ih povezuje. Zato metod dobro prepoznaje spirale, prstenove i izdužene oblike. Za razliku od njega, metode kao što su k-means, complete-linkage i average-linkage favorizuju kompaktnije klastere, zbog čega teže prepoznaju ovakve nekonveksne i lančane strukture.

Metod je takođe prirodno povezan sa minimalnim razapinjućim stablom, što omogućava jasnu i efikasnu implementaciju u kojoj se rastojanja između tačaka računaju po potrebi.

Poznat nedostatak metode je efekat ulančavanja (engl. *chaining effect*). Nekoliko tačaka koje predstavljaju šum može napraviti niz kratkih veza između dve inače odvojene grupe. Single-linkage ih tada može spojiti u jedan klaster, iako takva podela možda nije intuitivna.

# 3 Veza između single-linkage metode i MST-a

## 3.1 Grafovski model podataka

Neka je dat skup tačaka $p_1, \ldots, p_n$. Posmatra se kompletan težinski graf u kome je svaka tačka jedan čvor, a težina grane $(i, j)$ jednaka je euklidskom rastojanju $d(p_i, p_j)$ (mogla se koristiti i bilo koja druga metrika). Kompletan graf ima

$$\frac{n(n-1)}{2} = \Theta(n^2)$$

grana. Program ne mora da čuva sva rastojanja u memoriji, već se rastojanje između dve tačke računa tek kada je ono potrebno.

### 3.2 Dobijanje klastera iz MST-a

**Tvrdnja.** Ako se nad tačkama izgradi minimalno razapinjuće stablo i iz njega ukloni $k-1$ grana najveće težine, preostale komponente povezanosti daju single-linkage podelu na $k$ klastera.

Obrazloženje se najlakše vidi preko Kruskalovog algoritma. On razmatra grane po neopadajućoj težini i spaja dve komponente najlakšom dostupnom granom. Single-linkage radi istu stvar na nivou klastera: spaja dve komponente između kojih postoji najkraća veza. Redosled spajanja zato odgovara redosledu grana u MST-u.

Ako želimo $k$ klastera, postupak se zaustavlja pre poslednjih $k-1$ spajanja. Ekvivalentno tome, iz gotovog MST-a uklanja se $k-1$ najtežih grana.

### 3.3 Izbor Primovog algoritma

Eksplicitna konstrukcija kompletnog grafa zahtevala bi $\Theta(n^2)$ memorije. Kruskalov algoritam bi zatim sortirao $\Theta(n^2)$ grana, što bi koštalo $O(n^2 \log n)$ vremena. U implementaciji se koristi Primov algoritam prilagođen gustom, implicitnom grafu. Za svaki čvor čuvaju se najbolja poznata veza ka trenutnom stablu i roditelj preko koga je ta veza ostvarena. U svakom koraku linearno se pronalazi sledeći čvor, a zatim se ažuriraju rastojanja do svih neposećenih čvorova.

Ako je broj osobina $d$, računanje jednog rastojanja košta $O(d)$, pa ukupno vreme izgradnje MST-a iznosi

$$O(n^2 d)$$

Ne koristimo standardnu implementaciju Primovog algoritma pomoću reda sa prioritetom, jer je ona za guste grafove manje efikasna od implementacije sa linearnom pretragom.

## 4 Implementacija

U ovom poglavlju objasnićemo tok izvršavanja programa, kao i strukture i funkcije korišćene u njemu.

### 4.1 Tok programa

Program izvršava sledeće korake:

1. učitavanje zaglavlja i redova CSV fajla;
2. pronalaženje izabranih numeričkih kolona (osobina);
3. formiranje tačaka i njihovih identifikatora;
4. normalizacija svake osobine;
5. konstrukcija MST-a Primovim algoritmom;
6. uklanjanje $k-1$ najtežih grana;

7. određivanje komponenti pomoću DSU strukture;

8. ispis pregleda i čuvanje rezultata.

Program se pokreće iz komandne linije navođenjem ulaznog CSV fajla, željenog broja klastera i osobina koje učestvuju u klasterovanju:

```
./a.out data.csv broj_klastera lista_osobina [id_kolona|-] [max_redova]
```

Prvi argument predstavlja putanju do ulaznog CSV fajla, drugi željeni broj klastera, a treći listu numeričkih osobina odvojenih zarezima. Opciono se može navesti kolona koja služi kao identifikator tačke. Ako ID kolona nije potrebna, na njenom mestu može se proslediti znak `-`. Poslednjim opcionim argumentom ograničava se maksimalan broj redova koji će biti učitan. Rezultat klasterovanja čuva se u fajlu `output.csv`.

### 4.2 Strukture `Point` i `MstEdge`

Struktura `Point` predstavlja jednu tačku skupa podataka. Ona čuva vektor numeričkih osobina i tekstualni identifikator tačke. Struktura `MstEdge` predstavlja jednu granu minimalnog razapinjućeg stabla i čuva indekse njenih krajeva i težinu.

### 4.3 Klasa `DSU`

DSU (engl. *Disjoint Set Union*) služi za održavanje komponenti povezanosti nakon uklanjanja najtežih grana MST-a.

`find_parent` Funkcija pronalazi koren komponente kojoj pripada dati element. Pri rekurzivnom povratku primenjuje kompresiju putanje, čime ubrzava naredne pretrage iste komponente.

`unite` Funkcija spaja komponente koje sadrže dva prosleđena elementa. Ako su njihovi koreni različiti, stablo manjeg ranga vezuje se ispod stabla većeg ranga, a zatim se ažuriraju rang i veličina nove komponente.

`get_size` Funkcija vraća broj elemenata u komponenti kojoj pripada dati element.

### 4.4 `trim`

Funkcija uklanja beline sa početka i kraja stringa. Koristi se da bi nazivi kolona i vrednosti iz CSV fajla bili pravilno obrađeni i kada oko njih postoje suvišni razmaci.

### 4.5 `commaSplit`

Funkcija deli string na delove koristeći zarez kao separator i nad svakim delom poziva `trim`. Njome se obrađuju zaglavlje CSV-a, redovi podataka i argumenti sa imenima osobina.

### 4.6 `tryDouble`

Funkcija pokušava da ceo prosleđeni string pretvori u vrednost tipa `double`. Ako je konverzija uspešna, rezultat se upisuje kroz referencu `val` i vraća `true`; u suprotnom vraća `false`.

### 4.7 `buildColumnMap`

Funkcija prolazi kroz zaglavlje CSV fajla i pravi mapu koja svakom imenu kolone pridružuje njen indeks. Ostatak programa zato može da pronalazi kolone po imenu umesto po unapred zadatoj poziciji. Na primer, ako ovoj funkciji prosledimo

```
vector<string> header = {"ime", "godine", "visina", "hobi"};
```

ona će vratiti mapu:

```
{"ime" : 0, "godine" : 1, "visina" : 2, "hobi" : 3}
```

### 4.8 `resolveColumns`

Funkcija prima mapu kolona (dobijenu od `buildColumnMap`) i listu traženih imena osobina. Za svako ime pronalazi odgovarajući indeks i vraća vektor tih indeksa. Ako neka kolona ne postoji, baca izuzetak sa odgovarajućom porukom.

### 4.9 `readCSV`

Ovo je glavna funkcija za učitavanje podataka. Ona otvara fajl, čita zaglavlje, pronalazi osobine i opcionu ID kolonu, a zatim od svakog validnog reda pravi objekat `Point`. Ako ID kolona nije zadata, generišu se vrednosti oblika `row_0`. Redovi sa nenumeričkom vrednošću u nekoj izabranoj osobini preskaču se, dok parametar `maxRows` može da ograniči broj učitanih tačaka.

### 4.10 `normalize`

Funkcija standardizuje svaku osobinu zasebno. Prvo računa srednju vrednost $\mu$ i standardnu devijaciju $\sigma$, a zatim primenjuje formulu

$$x' = \frac{x - \mu}{\sigma}.$$

Ako je standardna devijacija nula, koristi se vrednost jedan. Funkcija menja prosleđene tačke, pa se kasnije u `output.csv` čuvaju normalizovane, a ne originalne vrednosti.

### 4.11 `euclideanDistance`

Funkcija računa euklidsko rastojanje između dve tačke. Dobijeno rastojanje predstavlja težinu grane u implicitnom kompletnom grafu.

### 4.12 `primMST`

Funkcija konstruiše minimalno razapinjuće stablo nad tačkama bez eksplicitnog čuvanja kompletnog grafa. Koristi nizove `inTree`, `best` i `parent`, koji redom označavaju da li je čvor već u stablu, težinu njegove najbolje poznate veze i roditelja preko koga je ta veza ostvarena.

Algoritam počinje od čvora 0. U svakom prolazu linearno pronalazi neposećen čvor sa najmanjom vrednošću `best`, dodaje odgovarajuću granu u MST i ažurira rastojanja do svih preostalih neposećenih čvorova. Rezultat je vektor od $n - 1$ objekata tipa `MstEdge`.

### 4.13 `getClusters`

Funkcija iz MST-a dobija zadati broj klastera. Proverava da li je $k$ ispravan, sortira grane od najteže ka najlakšoj i izostavlja prvih $k-1$. Krajeve preostalih grana spaja pomoću DSU strukture, a zatim svakoj dobijenoj komponenti dodeljuje identifikator klastera.

### 4.14 `printClusterSummary`

Funkcija grupiše tačke prema dobijenom klasteru, sortira klastere po veličini i za svaki ispisuje identifikator, broj članova i ID vrednosti prvih pet tačaka.

### 4.15 `saveCSV`

Funkcija u izlazni fajl upisuje kolone `id`, `cluster` i izabrane osobine. Za svaku tačku čuva njen ID, dobijeni klaster i numeričke vrednosti. Rezultat se čuva u fajlu `output.csv`, a osobine su normalizovane jer se funkcija poziva posle `normalize`.

### 4.16 `main`

Funkcija `main` povezuje sve delove programa. Proverava broj argumenata komandne linije, učitava njihove vrednosti, a zatim redom poziva `readCSV`, `normalize`, `primMST`, `getClusters`, `printClusterSummary` i `saveCSV`. Greške nastale u glavnom toku hvata `try-catch` blok.

## 5 Demonstracija programa na spiralnom skupu podataka

### 5.1 Skup podataka

Fajl `spiral.csv` sadrži 312 tačaka i kolone `x`, `y` i `label`. U klasterovanju učestvuju samo koordinate `x` i `y`. Kolona `label` koristi se kao identifikator radi naknadne provere, ali ne utiče na rastojanja niti na izbor klastera.

Na slici 1 prikazan je ulazni skup bez korišćenja labela.

### 5.2 Rezultat

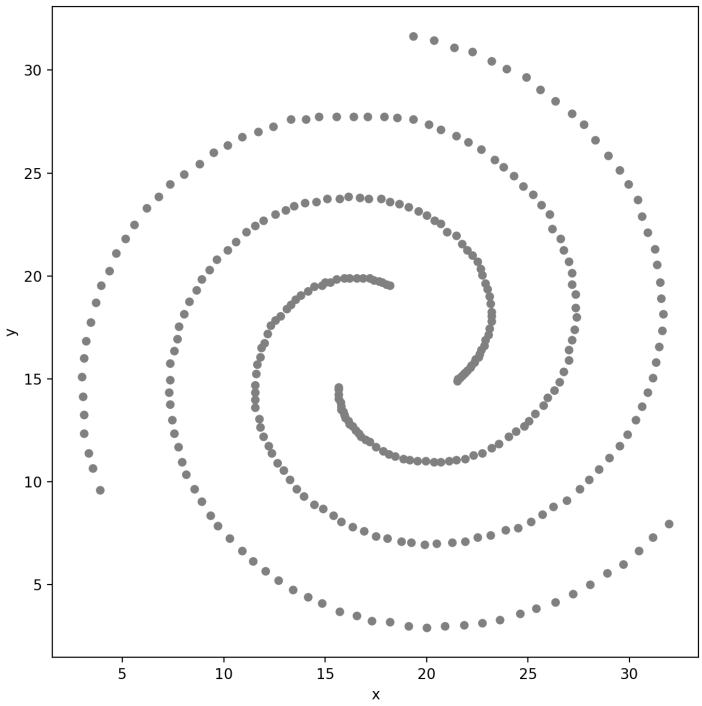
Za $k = 3$ program je odredio klastere veličina 106, 105 i 101. Provera prema koloni `label` pokazuje da svaki dobijeni klaster sadrži tačke samo jedne originalne labele.

**Napomena.** Ovo je demonstracioni primer, pa smo imali pristup stvarnim oznakama klastera; u realnim primenama one ne bi postojale.

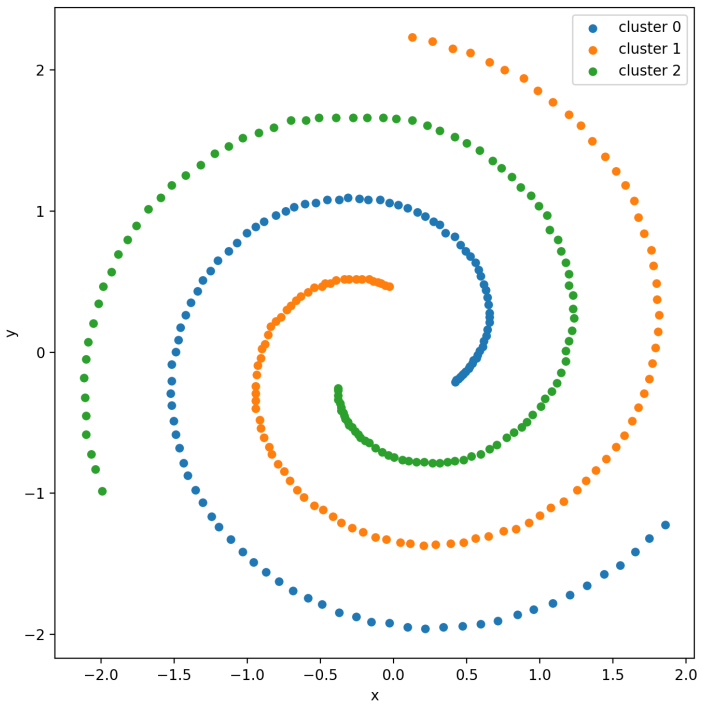
| Dobijeni klaster | Originalna labela | Broj tačaka |
|---|---|---|
| 0 | 3 | 106 |
| 1 | 1 | 101 |
| 2 | 2 | 105 |

Numeričke oznake klastera nemaju posebno značenje i zavise od redosleda kojim se obilaze DSU komponente. Važna je pripadnost tačaka, a ne sam broj kojim je klaster označen.

Slika 2 pokazuje da metod prati lokalnu povezanost tačaka duž spiralnih grana. Upravo je to slučaj u kome single-linkage ima prednost nad metodama koje favorizuju kompaktne i približno sferne klastere.




Slika 1: Spiralni skup pre klasterovanja




Slika 2: Rezultat single-linkage klasterovanja za $k = 3$.

## 6 Složenost i memorija

Neka je $n$ broj tačaka, a $d$ broj osobina. Učitavanje i normalizacija koštaju $\Theta(nd)$. Primov algoritam računa $\Theta(n^2)$ rastojanja, pa je njegova složenost $\Theta(n^2 d)$. Sortiranje $n-1$ grana MST-a košta $O(n \log n)$, a DSU operacije su skoro linearne.

Ukupna vremenska složenost je zato

$$O(n^2 d + n \log n)$$

odnosno $O(n^2)$ kada je $d$ mali fiksan broj.

Sam skup tačaka zauzima $O(nd)$ memorije. Pomoćni nizovi Primovog algoritma, MST i DSU zauzimaju $O(n)$ dodatne memorije. Zato je ukupna memorijska složenost $O(n)$ kada je $d$ mali fiksan broj.

## 7 Zaključak

Projekat pokazuje da single-linkage klasterovanje ima jasnu grafovsku interpretaciju. Minimalno razapinjuće stablo čuva veze potrebne za hijerarhijsko spajanje, a uklanjanjem $k-1$ najtežih grana dobija se podela na $k$ komponenti.

Implementacija obuhvata ceo tok obrade: učitavanje i pripremu podataka, računanje rastojanja, konstrukciju MST-a, formiranje klastera, čuvanje rezultata i vizuelizaciju. Spiralni skup jasno pokazuje prednost single-linkage metode na nekonveksnim oblicima: svih 312 objekata raspoređena su u tri klastera koji se poklapaju sa trima poznatim grupama iz skupa.

Gusta varijanta Primovog algoritma zahteva kvadratno vreme, ali ne čuva kompletan graf i koristi samo linearnu dodatnu memoriju. To predstavlja dobar kompromis između jednostavnosti, tačnosti i efikasnosti za skup veličine korišćene u demonstraciji.

## Literatura


[1] J. C. Gower i G. J. S. Ross, *Minimum Spanning Trees and Single Linkage Cluster Analysis*, Journal of the Royal Statistical Society, 1969.

[2] SciPy Developers, *scipy.cluster.hierarchy.linkage*, SciPy dokumentacija, docs.scipy.org.

[3] Wikipedia contributors, *Single-linkage clustering*, Wikipedia.